Fundamentals of Natural Language Processing

Chapter 1 — The NLP Landscape

Aymen Ben Brik

Chapter 1

The NLP Landscape

This opening chapter situates Natural Language Processing as a field: what problem it tries to solve, what makes that problem fundamentally hard, what canonical tasks define the discipline, and how its methods have evolved from hand-written rules to billion-parameter language models. Throughout, the goal is to develop a precise mental map you can return to as more advanced techniques are introduced.

1.1 What is NLP? Definition, scope, and core challenges

1.1.1 Motivation

Computers are excellent with *structured* data: spreadsheets, SQL tables, signed numerical vectors. They struggle with *unstructured* human language — the messy, fluid, ambiguous medium humans use to communicate. Yet a vast majority of the world's data exists in textual form: emails, chat messages, news articles, scientific papers, social-media posts, customer reviews, medical records, contracts. Building machines that can read, summarize, translate, search, and converse in natural language unlocks nearly every modern AI product, from search engines and recommendation systems to clinical decision-support tools and large language model assistants. *Natural Language Processing* (NLP) is the field that develops and studies the algorithmic foundations of this capability.

1.1.2 Approach by problem: the headline that everyone misreads

Problem. Consider the (real) news headline:

> "Stolen painting found by tree."

List every grammatically valid reading. How many require world knowledge to discard? How could an algorithm resolve the intended one?

Solution sketch. The headline admits at least two parses:

1. "Stolen painting [found by [tree]]" — a tree found the painting.
2. "Stolen painting [found [by tree]]" — the painting was found beside a tree.

Reading 1 is grammatically valid but pragmatically absurd: trees do not perform searches. A native speaker rules it out using world knowledge (trees are inanimate). A computer that does *not* have such knowledge cannot rule it out without an explicit mechanism — whether a hand-coded rule, statistical prior, or learned representation. Multiplying this difficulty across millions of sentences, in dozens of languages, is the central challenge NLP must address.

1.1.3 Definition and scope

Definition 1.1.1 (Natural Language Processing)**. Natural Language Processing** (NLP) is the subfield of artificial intelligence concerned with enabling computer systems to process, understand, and generate human language. It sits at the intersection of *linguistics* (the scientific study of language), *computer science* (algorithms and data structures) and *machine learning* (data-driven model building).

It is convenient to organise the field along three complementary axes:

- **Natural Language Understanding (NLU)** — extracting meaning, structure, sentiment, intent, or entities from text. *Inputs* are texts, *outputs* are typically structured representations (labels, parse trees, knowledge-graph triples).
- **Natural Language Generation (NLG)** — producing fluent and coherent natural-language outputs from structured inputs (database rows, dialogue states, summaries of long documents).
- **Interaction** — dialogue systems and conversational agents, where understanding and generation operate in a closed loop with a user.

1.1.4 Why natural language is hard

Three orthogonal sources of difficulty pervade every NLP system.

1. Ambiguity

A single linguistic expression can correspond to multiple meanings or structures. Ambiguity arises at every level of analysis.

- **Lexical ambiguity** (polysemy or homonymy). A single word denotes several distinct concepts.
  - *bank*: financial institution vs. edge of a river.
  - *date*: calendar day vs. romantic appointment vs. fruit.
- **Syntactic ambiguity**. A single string can be parsed by several distinct grammatical structures.
  - "*I saw the man with the telescope.*" — I used the telescope, or the man had a telescope?
- **Semantic ambiguity**. A single syntactic structure admits several meanings.
  - "*The chicken is ready to eat.*" — the chicken will eat, or will be eaten?

- **Pragmatic ambiguity**. The intended communicative function is underdetermined by the literal content.
  - "*Can you pass the salt?*" is a polite request, not a yes/no inquiry about the listener's capabilities.

2. Context dependence

The meaning of an expression cannot be computed from its surface form alone — it depends on its surrounding linguistic context and on world knowledge.

- "*Apple released a new product.*" (the company) vs. "*Apple is healthy.*" (the fruit). The same string, two referents.
- Pronoun resolution: "*John told Bill that he had failed the exam.*" — who failed? Either reading is grammatical; only context decides.
- Sarcasm and irony: "*Oh great, another delay.*" may carry the opposite of its literal meaning.

3. Variability

Many distinct surface forms encode the same meaning, and natural language permits substantial creativity.

- **Synonymy and paraphrasing**: "*happy*", "*joyful*", "*content*", "*elated*" are near-synonyms with different connotations.
- **Spelling and orthographic variation**: "*colour*" / "*color*", typos, leetspeak, abbreviations.
- **Cross-domain shifts**: medical, legal, social-media, and academic text differ in vocabulary, syntax, and conventions.
- **Multilingualism and code-switching**: "*J'ai*" un meeting demain — many speakers freely mix languages.

Remark 1.1.1. These three challenges combine multiplicatively. An algorithm that handles ambiguity but not context will fail on the simplest pronoun resolution; one that handles context but not variability will not generalize from training to deployment data.

1.1.5 Worked examples

Example 1 (Lexical ambiguity — Easy). Identify the polysemous word in the following sentence and give at least two distinct readings:

She left her glasses on the table.

Solution. The polysemous word is *glasses*, which may refer to:

- eyewear (a pair of corrective spectacles), or
- drinking vessels (e.g. wine glasses).

World knowledge — e.g. that the speaker is myopic, or that the table held a meal — is needed to disambiguate.

Example 2 (Syntactic ambiguity — Medium). Resolve the ambiguity of "*He hit the boy with the bat*" under the following two contexts:

Context A: The previous sentence was "*He grabbed a wooden bat from the dugout.*" *Context B:* The previous sentence was "*He met the boy with the bat at the zoo.*"

Solution. In context A, "*with the bat*" attaches to the verb *hit* (instrumental adjunct): the bat is the weapon. In context B, "*with the bat*" attaches to the noun *boy* (modifier): the boy carried a bat (the animal). Same surface form, two parse trees, two meanings; the intended one is selected only via context.

Example 3 (Pragmatics over literal content — Hard). The utterance "*It is cold in here.*" can play several pragmatic roles. List three plausible interpretations and explain when each applies.

Solution.

1. **Literal assertion**: a factual statement about the temperature, expecting acknowledgement.
2. **Indirect request**: an implicit suggestion to close a window, turn up heating, or hand the speaker a sweater (when the speaker has the social standing to make a request).
3. **Sarcasm**: in a room that is unmistakably warm, the same string may serve as ironic commentary on, e.g., an awkward silence.

Selecting the correct reading requires modelling the speaker, the listener, the physical situation, and the broader cultural conventions of indirect speech — topics studied under the name of *pragmatics*.

1.1.6 Python snippet: probing word senses

The Princeton WordNet lexical database, accessible through `NLTK`, allows us to enumerate distinct senses of a polysemous word.

```
import nltk
nltk.download('wordnet', quiet=True)
from nltk.corpus import wordnet as wn

def show_senses(word):
    senses = wn.synsets(word)
    print(f"'{word}' has {len(senses)} sense(s) in WordNet:")
    for syn in senses[:6]:
        print(f"  - {syn.name():<20s} {syn.definition()}")

show_senses("bank")
show_senses("date")
show_senses("light")
```

Running this snippet illustrates that even commonplace words carry many distinct senses — the raw input to any system that aspires to "understand" text.

1.1.7 Exercises

1. Find three polysemous English words used in everyday speech, and list at least two distinct senses of each. For each pair of senses, propose a sentence in which only one of them is felicitous.

2. For the sentence *"The duchess was entertaining last night,"* identify two grammatical readings and explain how the ambiguity is structured (lexical or syntactic).

3. Find a real-world headline (e.g. from the Wikipedia article on "Crash blossom" or from a news aggregator) that exhibits at least one form of ambiguity. Classify the ambiguity (lexical / syntactic / semantic / pragmatic) and propose a context that resolves it.

4. Discuss: how might *world knowledge* help a system disambiguate "*I deposited the money in the bank*"? What form would such knowledge take, and where might it be obtained at scale?

5. (Synthesis.) Suppose you must build a customer-service chatbot for a Tunisian e-commerce site that receives messages mixing French, Arabic and English. Identify, for each of the three challenges (*ambiguity*, *context*, *variability*), one concrete obstacle your system will face that an English-only system would not.

1.2 Major NLP tasks and applications

1.2.1 Motivation

NLP is not a single problem; it is a constellation of problems, each with its own *input format*, *output structure*, *evaluation metric*, and *characteristic methods*. A sentiment classifier and a machine-translation system both consume text, but they belong to different computational categories and therefore demand different architectures and training data. Practitioners must develop a clear taxonomy of NLP tasks for three practical reasons:

1. to *frame* a real-world business problem as the right computational task;

2. to *select* the appropriate model family (classification vs. sequence labelling vs. generation);

3. to *evaluate* models with metrics that match the task — accuracy is meaningful for classification but disastrous as a sole metric for translation.

1.2.2 Approach by problem: one dataset, many tasks

Problem. You receive a corpus of 50,000 customer reviews of a hotel chain. Each review is a paragraph of free text, sometimes with a star rating, sometimes not. Your manager asks: "What can we do with this data?" List at least five distinct NLP problems you could solve from *the same* raw input.

Solution sketch. The same corpus can power multiple downstream tasks:

1. **Sentiment analysis** — classify each review as positive, negative, or neutral.

2. **Topic classification** — assign reviews to themes (food, cleanliness, location, staff).

3. **Aspect-based sentiment** — extract sentiment per aspect ("rooms: positive, breakfast: negative").
4. **Named entity recognition** — extract place names, dates, hotel branches mentioned.
5. **Summarization** — produce a one-sentence digest of each review.
6. **Translation** — translate Arabic and French reviews into English for cross-lingual analytics.
7. **Question answering** — support a query interface ("What do guests say about the pool?").

Each item belongs to a different computational category, requires different annotated data, and is evaluated with different metrics. The remainder of this section gives a structured tour of these categories.

1.2.3 A taxonomy by input–output structure

We organise NLP tasks by what the model *consumes* and *produces*.

Category	Input	Output
Text classification	text	one (or several) discrete labels
Sequence labelling	text	one label per token
Sequence-to-sequence	text	a different text
Question answering	question + context	answer (span or text)
Open-ended generation	prompt	arbitrary text

The categories form a rough *spectrum of output complexity*, from a single integer (classification) to a free-form sequence (generation). Each step adds expressive power and evaluation difficulty.

1.2.4 Text classification

Definition 1.2.1 (Text classification)**.** Given an input text x drawn from a domain $\mathcal{X}$ and a finite label set $\mathcal{Y} = \{y_1, \ldots, y_K\}$, learn a function $f : \mathcal{X} \to \mathcal{Y}$ (single-label) or $f : \mathcal{X} \to \mathcal{P}(\mathcal{Y})$ (multi-label) that assigns labels to texts.

Canonical applications.

- **Sentiment analysis** — $\mathcal{Y} = \{\text{positive}, \text{negative}, \text{neutral}\}$. Powers product-review analytics, social-media monitoring, financial-sentiment trading signals.
- **Spam detection** — $\mathcal{Y} = \{\text{spam}, \text{ham}\}$. The earliest commercially deployed NLP application at scale.
- **Topic classification** — assign a document to one of several thematic categories (politics, sports, technology, ...).
- **Intent detection** — in chatbots, classify a user utterance into pre-defined intents (BOOK-FLIGHT, CANCEL-ORDER, ASK-STATUS).
- **Toxicity / hate-speech detection** — moderate user-generated content on social platforms.

Standard metrics. Accuracy, Precision, Recall, F_1 (Section **??** will define them precisely).

1.2.5 Sequence labelling

Definition 1.2.2 (Sequence labelling)**.** Given a sequence of tokens $x = (x_1, x_2, \ldots, x_n)$, predict a sequence of labels $y = (y_1, y_2, \ldots, y_n)$ of the same length, where each y_i belongs to a finite label set $\mathcal{Y}$.

The output dimension grows with the input, in contrast to classification.

Canonical applications.

- **Part-of-speech (POS) tagging** — label every token with its grammatical role: *noun, verb, adjective, ...*

  "*Apple/NOUN released/VERB a/DET new/ADJ phone/NOUN.*"

- **Named Entity Recognition (NER)** — label tokens that participate in named entities (persons, organizations, locations, dates) with their type and span.

  "*Apple/B-ORG was/O founded/O by/O Steve/B-PER Jobs/I-PER in/O Cupertino/B-LOC.*"

  The `B-`/`I-`/`O` scheme distinguishes *Begin* of an entity, *Inside*, and *Outside*.

- **Chunking and shallow parsing** — group tokens into noun phrases, verb phrases.

- **Slot filling** — in dialogue systems, identify the DATE, ORIGIN, DESTINATION slots in user utterances.

High-impact applications. Pharmacovigilance (drug-name extraction from medical records), legal-document analysis (clause boundaries), bioinformatics (gene and protein-name recognition), search-engine query understanding.

1.2.6 Sequence-to-sequence

Definition 1.2.3 (Sequence-to-sequence task)**.** Given a source sequence $x = (x_1, \ldots, x_n)$, produce a target sequence $y = (y_1, \ldots, y_m)$ where m may differ from n and the target vocabulary may differ from the source.

This category is the most general; it subsumes any task where the output is itself a piece of text.

Canonical applications.

- **Machine translation** — map a sentence in source language L_1 to its translation in target language L_2. Output length, word order, and idioms differ from the source.

- **Abstractive summarization** — produce a short summary that need not reuse the exact wording of the input. Contrast with *extractive* summarization, which selects sentences from the input verbatim.

- **Paraphrasing and style transfer** — rewrite a text with the same meaning but different surface form (e.g. formal to informal).
- **Grammar correction** — map an error-laden sentence to its corrected version.

Standard metrics. BLEU (translation), ROUGE (summarization). Both are studied formally in Section **??**.

1.2.7 Question answering

Definition 1.2.4 (Question answering)**.** Given a question q and (optionally) a context document or knowledge base c, produce an answer a to the question.

QA is conceptually a special case of sequence-to-sequence but is distinguished by the structure of the input (question + context) and by several distinct sub-paradigms:

- **Extractive QA**. The answer is a contiguous span of the context document. The model's job is to predict the start and end positions. Typical of reading-comprehension benchmarks (SQuAD).
- **Closed-book QA**. No context is provided; the model must rely on knowledge stored in its parameters.
- **Open-domain QA**. The model first *retrieves* relevant documents from a large corpus, then reads them to extract the answer. This is the foundation of the *retrieval-augmented generation* (RAG) pattern.
- **Generative QA**. The answer is generated freely, possibly synthesising information from the context.

Industrial deployments. Customer-service chatbots (intent+slot extraction), Web search (featured snippets), enterprise document search.

1.2.8 Open-ended generation

Definition 1.2.5 (Open-ended generation)**.** Given a prompt x, produce a continuation y that is fluent, coherent, and consistent with x but otherwise underdetermined.

This is the regime of large language models. The task is intentionally under-specified: many distinct continuations may be acceptable. Evaluation is therefore especially challenging.

Applications. Creative writing assistance, code generation (autocomplete and full-function synthesis), dialogue agents, automatic e-mail drafting, automated report generation from structured data.

1.2.9 Worked examples

Example 1 (Task framing — Easy). You are given the following row from a dataset of medical encounter notes:

> *Patient (45 y, F) reports persistent headache and was prescribed Paracetamol 500 mg twice a day.*

Identify two distinct NLP tasks that operate on this single sentence, naming the appropriate category for each.

Solution.

- **Named Entity Recognition** (sequence labelling): extract `[Drug: Paracetamol]`, `[Dose: 500 mg]`, `[Frequency: twice a day]`, `[Symptom: headache]`.
- **Text classification**: classify the note's clinical specialty (e.g. *neurology*).

Example 2 (Choosing the right output structure — Medium). You wish to build a system that, given a research-paper abstract, produces "the three most important takeaways". Should you frame this as a sequence-to-sequence task or as a classification task? Justify.

Solution. A classification task forces the output into a finite, pre-defined set of labels — but the "takeaways" are free-form sentences whose content depends on each abstract. The natural framing is therefore *abstractive summarization* (sequence-to-sequence), with an additional length constraint that forces the output to consist of three short sentences. A pre-defined-label classification approach would be tractable only if you replaced "the three most important takeaways" by "one of K predefined research-area tags," which is a substantively different problem.

Example 3 (Decomposing a complex query — Hard). A user asks the chatbot of a Tunisian airline: "*I bought a ticket to Paris last week, but my passport just expired. What should I do?*" Decompose this single utterance into the NLP sub-tasks the system must execute to respond usefully.

Solution.

1. **Intent detection** (classification): the intent is approximately HELP-WITH-DOCUMENT-ISSUE or CHANGE-BOOKING.
2. **Slot filling / NER** (sequence labelling): extract `[Destination: Paris]`, `[BookingDate: last week]`, `[DocumentIssue: expired passport]`.
3. **Knowledge retrieval** (open-domain QA): fetch the airline's policy on travel-document issues.
4. **Generation**: synthesise an empathetic, accurate, and fluent response that explains the available options (refund, ticket transfer, document renewal).

A modern dialogue system performs these steps either as separate components in a pipeline, or jointly inside a single large language model that is prompted with the user message and policy documents.

1.2.10 Python snippet: spaCy pipeline

The library `spaCy` ships pre-trained models that perform several of the tasks above in a single forward pass.

```python
# pip install spacy && python -m spacy download en_core_web_sm
import spacy
nlp = spacy.load("en_core_web_sm")
text = ("Apple was founded by Steve Jobs in Cupertino in 1976. "
        "Today, the company is worth over $3 trillion.")
doc = nlp(text)

print("Tokens with POS tags:")
for token in doc[:10]:
    print(f"  {token.text:<12s} {token.pos_:<6s} {token.dep_}")

print("\nNamed entities:")
for ent in doc.ents:
    print(f"  {ent.text:<20s} -> {ent.label_}")
```

A single call returns tokenisation, POS tags, dependency parse, and named entities — a concrete instantiation of the sequence-labelling and classification ideas of this section.

1.2.11 Exercises

1. For each of the following, identify the most appropriate NLP task category (classification, sequence labelling, sequence-to-sequence, QA, generation):

   (a) Detecting whether an SMS is fraudulent.

   (b) Translating a Tunisian Arabic Facebook post into French.

   (c) Extracting the names of all medications mentioned in a pharmacist's note.

   (d) Producing a one-paragraph summary of a court ruling.

   (e) Determining the writer's age range from a blog post.

2. For *aspect-based sentiment analysis* (e.g. "the food was great but the service was slow"), argue that the natural framing combines two distinct task categories. Which two?

3. Choose one industry (healthcare, finance, legal, education) and list three concrete NLP applications it uses. For each, identify the input, the output, and the dominant challenge from Section 1.1.

4. Discuss: should a search-engine query understanding system be framed as classification, sequence labelling, or generation? Defend your choice and identify trade-offs.

5. (Open-ended.) Sketch a pipeline that, given a Tunisian e-commerce review written in code-mixed French–Arabic, produces *(a)* an overall sentiment, *(b)* a list of mentioned product aspects, and *(c)* an English summary. Identify each NLP task involved and the order of execution.

1.3 NLP within the AI ecosystem

1.3.1 Motivation

Modern NLP is, in practice, a sub-discipline of *machine learning* (ML), which is itself a sub-discipline of *artificial intelligence* (AI). Reading any contemporary NLP paper requires fluency in concepts that originate outside NLP proper: gradient descent, regularization, train/validation/test splits, the bias–variance trade-off, the empirical risk minimization principle. Conversely, modern NLP has pushed the entire ML field forward: Transformers, scaling laws, instruction tuning, chain-of-thought prompting — innovations born in NLP now drive computer vision, speech, and robotics. Understanding where NLP sits in the broader AI landscape is therefore essential, both to use existing tools correctly and to situate new methods as they appear.

1.3.2 Approach by problem: the same data, three paradigms

Problem. Suppose you have collected 20,000 tweets about Tunisian banks. Describe how you could exploit this single corpus under three radically different learning regimes:

1. one in which every tweet has a hand-annotated sentiment label;
2. one in which no annotations are available;
3. one in which no annotations are available, but you wish to learn *useful representations* of words that subsequent tasks can re-use.

Solution sketch.

1. With labels: train a *supervised* sentiment classifier — input tweet, output positive / negative / neutral.
2. Without labels: *cluster* the tweets by topic; discover latent themes (*unsupervised*).
3. Without labels but for representation learning: train a model that, for each word, predicts the surrounding words in a sliding window (*self-supervised*). The model never receives "labels" from a human; it manufactures its own training signal from the data itself. This is exactly the principle behind Word2Vec (Section **??**) and, at much larger scale, behind every Large Language Model.

1.3.3 Situating NLP: AI, ML, DL

Definition 1.3.1 (Artificial intelligence)**. Artificial intelligence** (AI) is the area of computer science concerned with building systems that perform tasks that, when performed by humans, would be said to require *intelligence* — such as perception, reasoning, learning, language understanding, problem-solving, and action under uncertainty.

Definition 1.3.2 (Machine learning)**. Machine learning** (ML) is the sub-area of AI in which the system's behaviour is *induced from data* rather than directly programmed by humans. A learning algorithm transforms a dataset D into a predictor f.

Definition 1.3.3 (Deep learning)**. Deep learning** (DL) is the sub-area of ML that uses *deep neural networks* — parameterised functions composed of many layers of differentiable transformations — as the predictor family. Deep learning powers most state-of-the-art NLP, computer-vision, and speech systems.

The relationships between these fields are strictly inclusive: DL $\subset$ ML $\subset$ AI. NLP is a *domain of application*: it draws on rule-based AI for early systems, on statistical ML for the n-gram era, and on deep learning for everything since 2013.

1.3.4 The four ingredients of every learned NLP system

Definition 1.3.4 (Learning ingredients)**.** A supervised (or self-supervised) learning system is fully specified by the quadruple

$$(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$$

where:

- **Data** $D = \{(x_i, y_i)\}_{i=1}^{N}$ — a set of training examples, each consisting of an input and a target;
- **Model family** $\mathcal{F} = \{f_\theta : \theta \in \Theta\}$ — a parameterised set of functions from inputs to predictions;
- **Loss** $\mathcal{L}(\hat{y}, y)$ — a scalar measure of how badly a prediction $\hat{y}$ matches the target y;
- **Optimizer** $\mathcal{O}$ — an algorithm that updates θ to reduce the average loss on the data.

Definition 1.3.5 (Empirical risk minimization)**.** The dominant training principle is **empirical risk minimization** (ERM): seek the parameters θ^* that minimise the average loss on the training data,

$$\theta^* \in \arg\min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f_\theta(x_i),\, y_i\big).$$

Computing θ^* exactly is intractable for neural networks; the optimizer $\mathcal{O}$ (typically Stochastic Gradient Descent or its variants Adam, AdamW) approximates the minimum by iterative updates.

Remark 1.3.1. The four ingredients are useful as a debugging checklist. If a deployed system underperforms, the cause is almost always traceable to one of them: poor data quality, an inadequate model family, a mismatched loss, or an unstable optimization procedure.

1.3.5 Three learning paradigms

Supervised learning

Definition 1.3.6 (Supervised learning)**.** Each training example is a pair (x, y) where y is provided by an external annotator (the "supervision"). The learner's goal is to predict y from x on unseen inputs.

NLP examples. Sentiment classification, machine translation (parallel sentences as labels), POS tagging, NER. The bottleneck is annotation cost: linguistically reliable labels typically cost \$0.10–\$10 per example.

Unsupervised learning

Definition 1.3.7 (Unsupervised learning)**.** Training examples consist only of inputs x, with no labels. The goal is to discover *structure* in the data: clusters, latent variables, low-dimensional manifolds.

NLP examples. Topic modelling (LDA), document clustering, anomaly detection in log files. Output is structural rather than predictive.

Self-supervised learning

Definition 1.3.8 (Self-supervised learning)**.** Training examples consist only of inputs x, but the learner constructs a *pretext task* from the data itself — for example, predicting a hidden portion of x from the rest. Labels are manufactured automatically from the data, eliminating the need for human annotation.

Canonical NLP pretext tasks.

- **Causal language modelling**: given the prefix $w_1, \ldots, w_{t-1}$, predict the next token w_t. Used by GPT-style models.
- **Masked language modelling**: randomly mask $\sim 15\%$ of tokens, predict them from the surrounding context. Used by BERT.
- **Skip-gram and CBOW**: predict context words from a centre word, or vice-versa. Used by Word2Vec.

Remark 1.3.2 (Why self-supervision dominates modern NLP). Self-supervised pretraining converts *any* large unlabelled text corpus (a substantial fraction of the entire Web) into a training signal. The resulting models acquire general-purpose linguistic competence that is then *transferred* to downstream tasks via fine-tuning or prompting. This decoupling of *representation learning* (cheap, vast) and *task adaptation* (cheap, small) explains the dramatic progress of NLP since 2018.

1.3.6 Comparison table

Paradigm	**Labels?**	**Goal**	**NLP example**
Supervised	Provided by humans	Predict y from x	Spam detection
Unsupervised	None	Discover structure	Topic clustering
Self-supervised	Manufactured from x	Learn representations	Next-token prediction (LLMs)

1.3.7 Worked examples

Example 1 (Identifying the paradigm — Easy). For each of the following NLP tasks, identify the dominant learning paradigm:

1. Detecting whether a customer review is positive or negative, given a labelled dataset of 10,000 reviews.

2. Discovering the principal themes in 1 million unlabelled news articles.

3. Pretraining a 1B-parameter language model on 300 GB of Web text.

Solution. (1) Supervised. (2) Unsupervised (typically LDA or K-means on document embeddings). (3) Self-supervised (next-token or masked-token prediction).

Example 2 (The four ingredients in action — Medium). Spell out $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$ for a logistic-regression sentiment classifier trained on bag-of-words features.

Solution.

- $D = \{(x_i, y_i)\}_{i=1}^{N}$ where $x_i \in \mathbb{R}^V$ is a count vector over a vocabulary of size V, and $y_i \in \{0, 1\}$.
- $\mathcal{F} = \left\{ f_\theta(x) = \sigma(\theta^\top x) \,:\, \theta \in \mathbb{R}^V \right\}$ where σ is the logistic function.
- $\mathcal{L}(\hat{y}, y) = -\big(y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big)$ (binary cross-entropy).
- $\mathcal{O}$ = Stochastic Gradient Descent (or Adam) on the average of $\mathcal{L}$ over D.

This single example contains every ingredient that scales, in essence, to a billion-parameter Transformer trained on the Web.

Example 3 (Pretext-task design — Hard). Suppose you wish to train a self-supervised model that learns useful representations of *Tunisian Arabic* text but you have no labels and no parallel translation data. Propose a concrete pretext task, identify the corresponding $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$, and discuss one risk specific to dialectal Arabic.

Solution sketch.

- **Pretext task.** Masked-token prediction: replace $\sim 15\%$ of tokens by a special `[MASK]`, train the model to recover them.
- **Data** D: a corpus of Tunisian Arabic from social media, news, and forums (e.g. scraped Tunisian dialect from forums and Twitter).
- **Model** $\mathcal{F}$: a Transformer encoder with sub-word tokenization (BPE).
- **Loss** $\mathcal{L}$: cross-entropy on the masked positions.
- **Optimizer** $\mathcal{O}$: AdamW with warm-up.
- **Risk specific to dialectal Arabic.** Orthographic instability: the same word may be spelled in several ways (Latin script "arabizi", native script with or without diacritics, code-switched fragments). The tokenizer must be trained on the actual data — a tokenizer pre-trained on Modern Standard Arabic will fragment dialect words badly.

1.3.8 Python snippet: train your first text classifier in 10 lines

The library `scikit-learn` provides a complete supervised pipeline.

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

# Tiny illustrative dataset (replace with your own)
texts = ["I loved the food and the staff",      # positive
         "The room was dirty and noisy",        # negative
         "Fantastic value , will return",       # positive
         "Worst stay of my life , never again"] # negative
labels = [1, 0, 1, 0]

clf = Pipeline([
    ("tfidf", TfidfVectorizer(ngram_range=(1, 2))),
    ("logreg", LogisticRegression()),
])
clf.fit(texts, labels)
print(clf.predict(["The breakfast was great", "Awful service"]))
```

This snippet instantiates each of the four ingredients: *data* (`texts`, `labels`), *model family* (TF-IDF + logistic regression), *loss* (the default cross-entropy of `LogisticRegression`), and *optimizer* (the default L-BFGS solver). The same recipe scales — with much larger data, deeper models, and different losses — to state-of-the-art systems.

1.3.9 Exercises

1. Place each method in its sub-discipline (DL $\subset$ ML $\subset$ AI): expert systems, decision trees, BERT, K-means clustering, A* search, GPT-4.

2. For each task, identify the most appropriate learning paradigm:

   (a) Building a chatbot with 500 examples of (question, answer) pairs;

   (b) Discovering five product-feature themes in 10,000 unlabelled customer reviews;

   (c) Pretraining a French–Arabic encoder on a Web crawl with no aligned data.

3. Specify the four ingredients $(D, \mathcal{F}, \mathcal{L}, \mathcal{O})$ for a NER tagger using a feed-forward neural network on word-level features.

4. Discuss: why does the empirical risk on the *training* set systematically underestimate the empirical risk on *unseen* data? What practical safeguards address this?

5. (Open-ended.) For a Tunisian-bank social-media monitoring use case, design a three-stage pipeline:

   (a) self-supervised pretraining of an embedding model on bank-related tweets;

   (b) supervised fine-tuning for sentiment classification using 1,000 hand-labelled tweets;

   (c) unsupervised clustering of negative tweets to identify recurring complaint themes.

Identify the inputs, outputs, paradigm, and metric for each stage.

1.4 Discriminative vs. generative models

1.4.1 Motivation

A foundational distinction in machine learning splits models into two families: those that learn the *conditional distribution* $P(y \mid x)$ of a label given an input (*discriminative*), and those that learn the *joint* or *marginal* distribution $P(x, y)$ or $P(x)$ (*generative*). The distinction is not academic — it determines what data you need, what you can do at inference time, and how you should evaluate the system. Crucially, every modern Large Language Model is a generative model that has learnt to solve discriminative tasks *by being prompted*; understanding this paradigm shift requires fluency with both families.

1.4.2 Approach by problem: classify or generate?

Problem. Given a corpus of customer reviews, contrast two systems:

1. System A: predicts the sentiment label of a new review.
2. System B: writes a new, plausible review.

Which probability distribution must each system represent? Could either be repurposed for the other's task?

Solution sketch. System A only needs the conditional distribution $P(\text{sentiment} \mid \text{review})$. It does not need to know what reviews look like in detail; it only needs the *decision boundary* between positive and negative reviews. System B, in contrast, needs $P(\text{review})$ — a model of what an arbitrary review looks like. Repurposing: System B can in principle answer the discriminative question by computing $P(\text{review} \mid \text{positive})/P(\text{review} \mid \text{negative})$ via Bayes, but System A cannot generate reviews; the generative model is therefore strictly more expressive (and harder to learn).

1.4.3 Discriminative models

Definition 1.4.1 (Discriminative model)**.** A **discriminative model** parameterises the conditional probability of the label given the input,

$$P_\theta(y \mid x), \qquad \theta \in \Theta.$$

At inference, predict $\hat{y} = \arg\max_y P_\theta(y \mid x)$ (or sample from the conditional, in soft-prediction settings).

Examples in NLP.

- Logistic regression for spam detection.
- Conditional random fields for NER.
- Encoder-only Transformers (BERT, RoBERTa, DeBERTa) fine-tuned for classification, NER, or extractive QA.

Strengths. Direct optimization of the quantity of interest (the label probability); typically less data-hungry than generative models for a fixed task; tends to outperform comparable generative models on discriminative tasks.

Limitations. Cannot generate text; cannot detect anomalous inputs (since $P(x)$ is unknown); usually requires labelled data.

1.4.4 Generative models

Definition 1.4.2 (Generative model)**.** A **generative model** parameterises a probability distribution over the input space (or jointly over input and label):

$$P_\theta(x) \quad \text{or} \quad P_\theta(x, y).$$

At inference, generative models can: *(i)* sample new x from P_θ; *(ii)* score the likelihood of an observed x; and *(iii)* via Bayes, derive a discriminative classifier $P_\theta(y \mid x) \propto P_\theta(x \mid y)P_\theta(y)$.

Examples in NLP.

- N-gram language models: $P(w_t \mid w_{t-n+1}, \ldots, w_{t-1})$.
- Hidden Markov Models: a generative process for sequences.
- Decoder-only Transformers (GPT, LLaMA, Claude, Gemini): autoregressive generative language models, $P(w_t \mid w_1, \ldots, w_{t-1})$, scaled to billions of parameters.
- Encoder–decoder Transformers (T5, BART): generative for sequence-to-sequence tasks (translation, summarization).

Strengths. Can generate; can score arbitrary text; can be re-purposed for many downstream tasks via *prompting*; admit unlabelled training data by self-supervision.

Limitations. Far more parameters and data are required; harder to evaluate; their conditional probabilities derived via Bayes are typically less calibrated than those of a directly trained discriminative model.

1.4.5 Comparison and the modern blur

Aspect	**Discriminative**	**Generative**
Distribution	$P(y \mid x)$	$P(x)$ or $P(x, y)$
Training data	Labelled (x, y) pairs	Often unlabelled x
NLP archetypes	BERT, logistic regression, CRF	GPT, n-grams, T5
Can predict labels	Yes, directly	Yes, via Bayes
Can generate text	No	Yes
Sample efficiency	Higher for the target task	Lower per task; reusable across tasks
Modern role	Specialised, fine-tuned models	Foundation models, prompted

Remark 1.4.1 (The modern blur). Until 2018, the discriminative–generative split correlated tightly with the model architecture: one trained a separate model for each task. Modern Large Language Models have eroded this distinction. A decoder-only Transformer such as GPT-4 is a generative model in its training objective (next-token prediction) but is routinely *prompted* to solve discriminative tasks: "*Classify the following review as positive or negative: ...* ". The model produces the answer as a continuation, exploiting its general linguistic competence. This paradigm — a single generative model handling the full task taxonomy — explains the centrality of LLMs in contemporary NLP.

1.4.6 The Bayes bridge

Proposition 1.4.1 (From generative to discriminative)**.** *Any generative model with* $P_\theta(x \mid y)$ *and a class prior* $P_\theta(y)$ *induces a discriminative classifier via Bayes' rule:*

$$P_\theta(y \mid x) = \frac{P_\theta(x \mid y)\, P_\theta(y)}{\sum_{y'} P_\theta(x \mid y')\, P_\theta(y')}.$$

Proof. Direct application of the definition of conditional probability: $P(y \mid x) = P(x, y)/P(x)$, with $P(x)$ obtained by marginalisation $\sum_{y'} P(x, y')$. The class-conditional decomposition $P(x, y) = P(x \mid y)P(y)$ completes the derivation. □

Remark 1.4.2. The bridge is one-directional: a discriminative model $P(y \mid x)$ does *not* suffice to recover $P(x)$, because the marginal is integrated over y in a way that loses information about specific x-shapes.

1.4.7 Worked examples

Example 1 (Identifying the family — Easy). Classify each model as discriminative, generative, or both:

1. Naive Bayes spam classifier;
2. BERT fine-tuned for NER;
3. GPT-4 used to generate marketing copy;
4. GPT-4 prompted to label movie reviews as positive/negative;
5. A Transformer trained to predict the next token of a code corpus.

Solution.

1. **Generative** — Naive Bayes models $P(x \mid y)P(y)$ explicitly, even though it is most often used for classification.
2. **Discriminative** — BERT fine-tuned for NER directly models P(tag-sequence | tokens).
3. **Generative**.
4. **Generative used discriminatively** — the underlying model is generative but it is being applied to a discriminative task via prompting.

5. **Generative**.

Example 2 (Bayes bridge in practice — Medium). A Naive Bayes spam classifier estimates from training data:

$$P(\text{spam}) = 0.2, \quad P(\text{ham}) = 0.8.$$

For a particular short message x, the model also estimates

$$P(x \mid \text{spam}) = 10^{-4}, \quad P(x \mid \text{ham}) = 5 \times 10^{-6}.$$

Compute $P(\text{spam} \mid x)$.

Solution.

$$P(\text{spam} \mid x) = \frac{0.2 \times 10^{-4}}{0.2 \times 10^{-4} + 0.8 \times 5 \times 10^{-6}} = \frac{2 \times 10^{-5}}{2 \times 10^{-5} + 4 \times 10^{-6}} = \frac{20}{24} \approx 0.833.$$

The message is classified as spam with $\approx 83\%$ confidence. Note that the prior $P(\text{spam}) = 0.2$ is much smaller than 0.833 — the class-conditional likelihood ratio shifts the posterior strongly.

Example 3 (Modern LLM as a discriminative substitute — Hard). You wish to build a sentiment analysis system for product reviews. You can choose between:

- (a) Fine-tuning a 110M-parameter BERT model on 5,000 labelled reviews;
- (b) Prompting a 100B-parameter generative LLM with the same 5,000 reviews as in-context examples.

Discuss three trade-offs.

Solution.

1. **Cost.** Option (a) requires GPU-hours for fine-tuning and a small inference-time footprint; option (b) requires zero training but expensive per-call inference (LLM API or large GPU).
2. **Sample efficiency / accuracy.** BERT fine-tuned on 5,000 examples typically attains higher accuracy on the target task than an LLM with 5,000 in-context demonstrations, because the discriminative architecture is directly optimised for the task. The LLM, however, requires no update to its parameters and can be redirected to a new task by editing the prompt.
3. **Maintenance and adaptation.** BERT must be re-fine-tuned each time the label set changes; the LLM accommodates new labels by prompt rewriting, at no parameter cost. In a fast-evolving product catalogue, this can dominate other considerations.

1.4.8 Python snippet: probabilistic interpretation

```python
import numpy as np
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import CountVectorizer

texts = [
    "great service highly recommend",    # positive
    "terrible food never again",         # negative
    "loved the room and the staff",      # positive
    "worst experience of my life",       # negative
]
labels = [1, 0, 1, 0]

X = CountVectorizer().fit_transform(texts)

# Generative: Multinomial Naive Bayes
gen = MultinomialNB().fit(X, labels)
print("Naive Bayes log P(x|y) and P(y):")
print("  log P(y)   :", gen.class_log_prior_)
print("  log P(x|y) shape:", gen.feature_log_prob_.shape)

# Discriminative: Logistic Regression
disc = LogisticRegression().fit(X, labels)
print("\nLogistic Regression P(y|x) on a held-out sentence:")
heldout = CountVectorizer(vocabulary=
            CountVectorizer().fit(texts).vocabulary_).fit_transform
               (
            ["the food was perfect"])
print("  P(y|x) :", disc.predict_proba(heldout))
```

The `class_log_prior_` and `feature_log_prob_` attributes of `MultinomialNB` are exactly the generative quantities $\log P(y)$ and $\log P(x \mid y)$; the `predict_proba` of `LogisticRegression` is the discriminative quantity $P(y \mid x)$.

1.4.9 Exercises

1. Classify each model as discriminative, generative, or hybrid:

   (a) Hidden Markov Model for POS tagging;

   (b) Conditional Random Field for NER;

   (c) Variational Auto-Encoder for text;

   (d) Decoder-only Transformer used as a chatbot;

   (e) Encoder-only Transformer used as a sentiment classifier.

2. Suppose a generative spam model gives $P(x \mid \text{spam}) = 2 \times 10^{-5}$, $P(x \mid \text{ham}) = 10^{-3}$, with priors $P(\text{spam}) = 0.3$, $P(\text{ham}) = 0.7$. Compute the posterior probability that x is spam.

3. Argue why a generative model used for classification can outperform a discriminative model when training data is scarce. (Hint: think about what the generative model can "learn from" beyond the labels.)

4. In the LLM-as-classifier paradigm, the model receives a prompt such as "*Classify: positive or negative? — This restaurant was excellent.*" Identify three sources of variance in the predicted label and propose a mitigation for each.

5. (Synthesis.) For a moderation system that must *(a)* classify a tweet as toxic / safe and *(b)* detect previously unseen forms of toxicity, argue why a hybrid discriminative + generative architecture could outperform either alone.

1.5 Historical evolution: from rules to large language models

1.5.1 Motivation

NLP has reinvented itself roughly once per decade. Each new era inherits the data and applications of its predecessor but discards the previous era's modelling assumptions. Understanding this recurring cycle pays off in two ways: it explains *why* certain methods exist (each is the answer to a previous limitation), and it provides a forecasting heuristic — when a modern method has a clear failure mode, the next era's method is likely to address exactly that mode. The pattern also highlights a single guiding principle: *at every step, hand-engineered structure has been replaced by learned representations from data, and the volume of data and parameters has grown by orders of magnitude.*

1.5.2 Approach by problem: how would you build a chatbot in 1966?

Problem. Without machine learning, neural networks, or even substantial computing power, how could you build a system that converses in natural language with a human user? Sketch the design Joseph Weizenbaum chose for ELIZA in 1966.

Solution sketch. Weizenbaum designed *pattern-matching templates* keyed on user input. A user message such as "*I am unhappy.*" triggers a script associated with the keyword *unhappy*, which produces a response such as "*Why are you unhappy?*". ELIZA performs no understanding; it shuffles surface forms by string substitution. Yet several users believed they were conversing with a human therapist — the so-called ELIZA effect. The strengths and limits of this pattern were the seeds of the next era: the system was *linguistically brittle, knowledge-bound, and domain-specific*, but proved that *some* useful interaction is achievable with extremely simple machinery.

1.5.3 Era 1 — Rule-based systems (1950s–1980s)

Approach. Linguists hand-write rules. Knowledge bases enumerate facts. Formal grammars (context-free, context-sensitive) describe syntax. Algorithms parse, transform, or generate strings according to the rules.

Representative systems.

- *Georgetown–IBM experiment* (1954): a Russian–English machine translator built from ∼250 rules and a 6-rule grammar. Demoed live to a delighted press; declared the imminent end of human translators.
- *ELIZA* (Weizenbaum, 1966): the pattern-matching "psychotherapist" described above.
- *SHRDLU* (Winograd, 1972): a system that conversed in English about a blocks world.
- Rule-based spell checkers, dictionary-based morphological analysers, regex-driven information extraction.

Limitations. Rules do not scale: every new domain requires new rules, every new language requires re-coding. Ambiguity is hard to capture in rule form (which reading does the rule choose?). Coverage is brittle: any input not anticipated by the rules fails silently. Maintenance costs grow super-linearly with the rule base.

1.5.4 Era 2 — Statistical NLP (1990s–2000s)

Approach. Replace hand-written rules by *statistical models* estimated from text corpora. Use probabilistic decision rules: among parse trees, output the most probable; among translations, output the most likely target sentence. The IBM Candide statistical machine translation system, developed in the late 1980s and early 1990s, set the tone.

N-gram language models. The era's flagship is the n-gram. Given a word sequence $w_1 \dots w_T$, factorize:

$$P(w_1, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1}) \approx \prod_{t=1}^{T} P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

The *Markov assumption* ($n-1$ words of context suffice) makes the model trivial to estimate by counting in a corpus:

$$\widehat{P}(w_t \mid w_{t-1}, \dots, w_{t-n+1}) = \frac{\#(w_{t-n+1}, \dots, w_t)}{\#(w_{t-n+1}, \dots, w_{t-1})}.$$

Other emblematic methods.

- Hidden Markov Models for POS tagging and speech recognition.
- Bag-of-Words and TF–IDF for text classification and information retrieval.
- Maximum-entropy / log-linear models for sequence labelling.

Limitations. Sparse data: most n-grams of order ≥ 4 never appear in any corpus. Smoothing techniques (Good–Turing, Kneser–Ney) patch the worst sparsity but do not solve the underlying problem. The context window is hard-coded and small. There is no notion of semantic similarity — "*cat*" and "*kitten*" are as unrelated to the model as "*cat*" and "*democracy*".

1.5.5 Era 3 — Neural networks and word embeddings (2013–2017)

Approach. Represent each word as a *dense vector* learnt from data. Words that appear in similar contexts receive similar vectors. The model thus generalises across surface forms.

Landmark contributions.

- *Word2Vec* (Mikolov et al., 2013): two shallow neural architectures (*skip-gram*, *CBOW*) train word vectors by predicting context from a centre word, or vice-versa. The vectors exhibit striking algebraic regularities: $\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}} \approx \overrightarrow{\text{queen}}$.
- *GloVe* (Pennington et al., 2014): global matrix factorization of the word–context co-occurrence matrix.
- *FastText* (Bojanowski et al., 2017): extends Word2Vec to sub-word units, handling out-of-vocabulary words and morphologically rich languages.
- *Sequence models* (LSTMs, GRUs): recurrent networks process sequences without the fixed n-gram window, partially solving the long-context problem.
- *Encoder–decoder architectures with attention* (Bahdanau et al., 2015): a neural translator can "*attend*" to all source positions when generating each target token.

Limitations.

- Word2Vec / GloVe / FastText produce *static* embeddings: "*bank*" has one vector, regardless of whether it refers to a financial institution or a riverside.
- Recurrent networks process sequences strictly left-to-right (or right-to-left), making long-range dependencies hard to capture and parallelisation difficult.

1.5.6 Era 4 — Transformers and large language models (2017–present)

Approach. A new architecture — the *Transformer* — replaces recurrence by *self-attention*, allowing every token to directly attend to every other token in the sequence. The Transformer decouples sequence length from depth and is fully parallelisable, enabling training on data and parameter scales unattainable for RNNs.

Three foundational papers.

- Vaswani et al. (2017), *"Attention Is All You Need"*: introduces the Transformer.
- Devlin et al. (2018), *BERT*: encoder-only Transformer pretrained by masked language modelling, then fine-tuned on downstream classification, NER, QA. State-of-the-art on dozens of benchmarks.
- Radford et al. (2018, 2019, 2020), *GPT-1, GPT-2, GPT-3*: decoder-only Transformer pretrained as a causal language model. Scaling demonstrated *emergent abilities* — in-context learning, instruction following, chain-of-thought reasoning — not present in smaller models.

Hallmarks of the era.

- **Contextual embeddings.** A token's representation depends on its surrounding context: "*bank*" in a financial sentence and in a river sentence receive different vectors.
- **Self-supervised pretraining at Web scale.** Models trained on hundreds of billions of tokens of unlabelled text.
- **Foundation models.** A single pretrained model is adapted (by fine-tuning, prompting, or retrieval augmentation) to dozens of downstream tasks.
- **Scaling laws** (Kaplan et al., 2020; Hoffmann et al., 2022): test loss decays predictably as a power law in parameters, data, and compute, providing a quantitative roadmap for model design.
- **Reasoning and tool use.** Modern LLMs can be prompted to produce intermediate reasoning steps (*chain-of-thought*), to call external tools, or to be aligned with human preferences via reinforcement learning from human feedback (RLHF).

1.5.7 Synthetic comparison

Era	Representation	Semantics	Scalability	Period
Rules	Symbolic	Manual	Low	1950–19
Statistical (BoW, n-grams)	Counts / sparse vectors	Surface only	Medium	1990–20
TF–IDF	Weighted counts	Partial	Medium	1990–20
Word2Vec / GloVe / FastText	Static dense vectors	Yes (static)	High	2013–20
Transformers / LLMs	Contextual dense vectors	Yes (contextual)	Very high	2017–pr

1.5.8 The recurring pattern

Remark 1.5.1. Each era addressed a specific failure of its predecessor:

- Rules → statistical: *rules don't scale or handle ambiguity probabilistically.*
- Statistical → neural: *counts cannot capture semantic similarity.*
- Static embeddings → Transformers: *a word's meaning depends on context.*

A productive question to ask of any present-day NLP system is: *what is its specific failure mode, and what kind of method could fix it?* The most likely answer points at the next era.

1.5.9 Worked examples

Example 1 (Era identification — Easy). Place each method in its era:

1. Pattern-matching chatbot;
2. Trigram language model with Kneser–Ney smoothing;
3. BERT fine-tuned for sentiment;

4. Word2Vec skip-gram embeddings;

5. Hand-coded regex-based information extractor.

Solution. (1) Rule-based; (2) Statistical; (3) Transformers / LLMs; (4) Neural / static embeddings; (5) Rule-based.

Example 2 (Identifying era from a failure mode — Medium). A deployed translation system from Modern Standard Arabic to French is fluent on news articles but produces incoherent translations on Tunisian dialect tweets. Which era's failure mode is this, and what era of method would address it?

Solution. The failure is a *distributional mismatch*: the training data does not cover the target dialect's surface forms, so the model fails to generalise. This is most consistent with a *static-embedding-era* (or earlier) limitation, because contextual Transformer-era models trained on dialect data would, in principle, handle the variability through sub-word tokenisation and contextualisation. The fix is therefore to *(a)* introduce sub-word tokenisation (FastText-/BPE-style) and *(b)* pretrain on dialect data — both Transformer-era techniques.

Example 3 (The bigram inheritance — Hard). Show that in the limit where the context window is reduced to one previous token, a Transformer language model degenerates into a bigram model.

Solution sketch. A Transformer language model defines $P(w_t \mid w_1, \ldots, w_{t-1})$ via attention over the entire history. If we restrict the model so that each token attends only to its immediate predecessor (a *causal mask of width* 1), the conditional becomes $P(w_t \mid w_{t-1})$. With sufficient capacity to memorise the bigram counts, the model would, at convergence, recover the same conditional probabilities as a bigram language model trained on the same corpus. This thought experiment shows that statistical and modern approaches lie on a spectrum of context-window choices; modern systems differ primarily in the *breadth and learned weighting* of context, not in their underlying probabilistic objective.

1.5.10 Python snippet: a four-line bigram language model

The simplest statistical-era artefact is illuminating to implement and evaluate.

```
from collections import Counter
import math, random

text = ("the␣cat␣sat␣on␣the␣mat␣the␣cat␣sat␣on␣the␣floor␣"
        "the␣dog␣sat␣on␣the␣mat␣the␣dog␣ran␣on␣the␣floor").split()

bigrams = Counter(zip(text, text[1:]))
unigrams = Counter(text)

def P(w_next, w_prev):
    """Conditional bigram probability with smoothing."""
    return (bigrams[(w_prev, w_next)] + 1) / (unigrams[w_prev] +
       len(unigrams))

def generate(start, n):
    seq = [start]
    for _ in range(n):
```

```
        candidates = [w for w in unigrams if (seq[-1], w) in
            bigrams]
        if not candidates:
            break
        weights = [P(w, seq[-1]) for w in candidates]
        seq.append(random.choices(candidates, weights=weights)[0])
    return "␣".join(seq)

print(generate("the", 8))
```

The same conceptual machinery — conditional probabilities, sampling — powers GPT-4, scaled by ten orders of magnitude.

1.5.11 Exercises

1. Date each method to its era (rules / statistical / static neural / Transformer):
   (a) Hidden Markov Model for speech recognition;
   (b) Convolutional neural network for sentiment classification;
   (c) Mistral 7B large language model;
   (d) Porter stemming algorithm;
   (e) TF–IDF cosine retrieval.
2. Why did n-gram models choose $n \leq 5$ in practice, despite the formal benefit of larger n? Identify the precise statistical phenomenon and the smoothing techniques designed to mitigate it.
3. Word2Vec "solves" the n-gram era's most visible failure (no semantic similarity). What new failure does it introduce, and which era addresses it?
4. Discuss: each era replaces *hand engineering* by *data-driven learning*. What aspect of NLP, in your judgement, is still hand-engineered today, and what kind of method might learn it?
5. (Open-ended forecasting.) Identify a clear failure mode of contemporary LLMs (e.g. hallucination, lack of calibrated uncertainty, poor multilingual coverage on low-resource languages such as Tunisian dialects). Speculate on the methodological direction the next era might take to address it.